

git

Prácticas — parte 2: gestión de ramas

Antes de empezar

- Abre una terminal. Todo se hace **en local**, no hace falta GitHub ni conexión a internet.
- Vas a crear un proyecto nuevo (un mini-portfolio) y a resolverlo con ramas, igual que en la teoría.
- Ve resolviendo los ejercicios en orden, cada uno se apoya en el anterior.
- **git reset** no se usa en ningún ejercicio. Si en algún momento quieres deshacer algo, usa `git checkout`, `git revert` o simplemente corrige a mano — es la norma de la asignatura.
- Los **retos** (al final) no traen los pasos, solo el objetivo: decides tú los comandos.

Ejercicios guiados

Ejercicio 1 — Prepara el proyecto

1. Crea una carpeta `practica-ramas`, entra en ella e inicializa un repositorio git.
2. Si tu rama inicial se llama `master` en vez de `main`, renómbrala ahora mismo (usa el comando que acabas de aprender en teoría).
3. Crea `index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Mi Portfolio</h1>

  <section id="sobre-mi">
    <h2>Sobre mí</h2>
    <p>Soy estudiante de Desarrollo de Aplicaciones Web.</p>
  </section>
```

Ejercicio 2 — Corrige tu propio commit (`--amend`)

1. Crea un fichero `README.md` con un par de líneas describiendo el proyecto.
2. Confírmalo a propósito con este mensaje, con una errata: `git commit -m "Añad README del proyecto"`.
3. Sin crear un commit nuevo, corrige el mensaje para que diga correctamente "Añado README del proyecto".

Resultado esperado: `git log --oneline` sigue mostrando 2 commits en total (no 3), y el mensaje del último ya está corregido.

Ejercicio 3 — Observa cómo se mueve **HEAD**

1. Ejecuta `git log --oneline` y fíjate en dónde aparece `(HEAD -> main)`.
2. Crea y cámbiate a una rama nueva en un solo paso: `git checkout -b fix/arreglo-sobre-mi`.
3. Vuelve a ejecutar `git log --oneline`.

Resultado esperado: ahora ves `(HEAD -> fix/arreglo-sobre-mi, main)` en el mismo commit. Explica con tus palabras qué ha cambiado y qué no.

Ejercicio 4 — Primera rama: `fix/arreglo-sobre-mi`

(sigues en esa rama desde el ejercicio anterior)

1. Corrige la errata de `index.html` : "studiente" → "estudiante".
2. Confirma el cambio con un commit.
3. Vuelve a `main`.
4. Fusiona: `git merge fix/arreglo-sobre-mi`.
5. Lee con atención el mensaje que te devuelve la terminal.

Resultado esperado: el mensaje incluye la palabra `Fast-forward`, porque `main` no había cambiado mientras tanto.

Ejercicio 5 — Segunda rama: `feature/seccion-contacto`

1. Crea y cámbiate a una rama nueva `feature/seccion-contacto` a partir de `main`.
2. Añade una sección nueva a `index.html`, justo después de "Proyectos":

```
<section id="contacto">
  <h2>Contacto</h2>
  <p>Puedes escribirme a mi@email.com</p>
</section>
```

3. Confirma el cambio.
4. **Antes de fusionar**, vuelve a `main` y añade tú, directamente ahí, un pequeño cambio (por ejemplo un comentario `<!-- Portfolio v2 -->` al principio de `index.html`). Confírmalo también.
5. Ahora fusiona `feature/seccion-contacto` en `main`.

Desplegare la aplicación (ver 5.0.0) `Result: Merge made by the 'ort'`

Ejercicio 6 — Limpieza de ramas

1. Lista las ramas actuales con `git branch`.
2. Borra `fix/arreglo-sobre-mi` (ya fusionada) con `-d`.
3. Borra `feature/seccion-contacto` (ya fusionada) igual.
4. Crea una rama de prueba `tmp-borrarme`, sin fusionarla, e intenta borrarla con `-d`.
¿Qué ocurre?
5. Bórrala ahora con `-D`.
6. Crea otra rama `tmp2`, renómbrala a `prueba-nombre` y luego bórrala.

Resultado esperado: entiendes cuándo `-d` te deja borrar una rama y cuándo hace falta `-D`.

Ejercicio 7 — Visualiza el historial

```
git log --graph --oneline --all
```

Busca en el resultado:

- El commit de fusión "de verdad" (con dos líneas confluyendo) del ejercicio 5.
- El commit del ejercicio 4, que **no** generó un commit de fusión propio.

Resultado esperado: sabes distinguir a simple vista, en el gráfico, un fast-forward de un merge de tres vías.

Ejercicio 8 — Recuperar un fichero de otra rama

1. Crea y cámbiate a `feature/renombrar-clases-css` a partir de `main`.
2. En `styles.css` e `index.html`, renombra las clases `.caja1` / `.caja2` a `.proyecto-card`.
3. Confirma el cambio.
4. Vuelve a `main` **sin fusionar todavía**.
5. Trae solo `styles.css` desde la otra rama, sin fusionar toda la rama:

```
git checkout feature/renombrar-clases-css -- styles.css
```

6. Comprueba con `git status` / `git diff` qué ha cambiado en `main`.
7. Deshaz ese cambio suelto para dejar `main` como estaba: `git checkout styles.css`.

Ejercicio 9 — Detached HEAD controlado

1. `git log --oneline` y copia el hash del **commit inicial** (ejercicio 1).
2. Haz `git checkout <ese_hash>`. Lee el aviso de "detached HEAD".
3. Modifica algo pequeño en `index.html` (por ejemplo, un comentario) y confírmalo **ahí mismo**, en detached HEAD.
4. Antes de moverte a ninguna parte, rescátalo creando una rama en este punto: `git checkout -b experimento-antiguo`.
5. Comprueba con `git log --oneline --all` que ese commit ya tiene una rama que lo señala.

Resultado esperado: entiendes por qué el paso 4 es imprescindible antes de cambiar de rama.

Ejercicio 10 — Provoca y resuelve un conflicto

1. Vuelve a `main` y fusiona `feature/renombrar-clases-css` (normal, sin conflicto).
2. Desde `main`, crea dos ramas nuevas: `feature/fondo-azul` y `feature/fondo-verde`.
3. En `feature/fondo-azul`, añade a `styles.css`: `body { background: lightblue; }`. Confirma.
4. En `feature/fondo-verde`, añade la **misma línea** en el **mismo sitio**, pero con `lightgreen`. Confirma.
5. Vuelve a `main` y fusiona `feature/fondo-azul` (debería ir bien).
6. Fusiona ahora `feature/fondo-verde`. Debería salir **CONFLICT**.
7. Abre `styles.css`, decide con qué color te quedas (o dejas los dos), elimina los marcadores `<<<<<<`, `=====`, `>>>>>>`.
8. `git add + git commit` para cerrar la fusión.

Ejercicio 11 — Deshacer con seguridad: `git revert`

1. Haz un commit "por error" que borre por completo la sección "Contacto" de `index.html`.
2. Imagina que ese commit ya lo ha descargado el resto del equipo — **no puedes usar `reset`**.
3. Deshazlo con `git revert <hash_del_commit_erróneo>`.
4. Comprueba con `git log --oneline` que hay un commit nuevo que deshace el anterior, y que el commit "erróneo" **sigue** en el historial.

Resultado esperado: la sección "Contacto" vuelve a aparecer en `index.html`, y el historial conserva ambos commits (el error y su corrección).

Retos

Aquí no hay pasos: piensa qué comandos necesitas.

Reto 1 — Ramas en cascada

Objetivo: crea una rama nueva a partir de **otra rama** que todavía no está fusionada en `main` (no partas de `main`). Haz un cambio, fusiónala primero en su rama "padre", y después fusiona esa rama "padre" en `main`.

Comprueba con `git log --graph --oneline --all` que el historial refleja esa cascada.

Reto 2 — El commit que se pierde

Objetivo: entra en detached HEAD sobre un commit antiguo, haz un commit ahí **sin** crear una rama de rescate, y cambia de rama.

Comprueba con `git log --oneline --all` que ese commit ya no aparece en ningún sitio. Explica por qué ha pasado.

Reto 3 — Revert del revert

Objetivo: a partir del ejercicio 11, revierte tu propio commit de revert.

¿Qué contenido tiene `index.html` después de este segundo revert? ¿Cuántos commits hay ahora relacionados con la sección "Contacto"?

Reto 4 — Nombra bien tus ramas

Imagina estas tres tareas:

- (a) hay un bug urgente en producción
- (b) quieres añadir un modo oscuro al portfolio
- (c) vas a preparar la versión 1.0 para entregar

Objetivo: crea las tres ramas con el nombre que les correspondería según la convención vista en clase (no hace falta trabajar realmente en ellas).

Reto 5 — Fuerza un merge de tres vías

Objetivo: consigue una fusión en la que `main` no pueda hacer fast-forward (ha avanzado mientras tanto) y que, aun así, termine **sin conflicto**.

Identifica en `git log --graph` el commit de fusión resultante.

Autoevaluación

¿Te ves capaz de...?

- [] Explicar qué es `HEAD` y cómo se mueve al hacer commits o cambiar de rama.
- [] Crear, listar, renombrar y eliminar ramas (`-d` y `-D`).
- [] Explicar la diferencia entre un merge *fast-forward* y uno de tres vías.
- [] Resolver un conflicto de merge de principio a fin, sin ayuda.
- [] Explicar qué es un *detached HEAD* y cómo rescatar un commit hecho ahí.
- [] Explicar la diferencia entre `git reset` y `git revert` , y por qué en clase usamos `revert` .
- [] Nombrar ramas siguiendo una convención (`feature/` , `fix/` ...).
- [] Leer un `git log --graph --oneline --all` y entender qué ha pasado.